

AI Capstone HW1

112550169 潘仰祐

1. Implementation

Task 1: Data Collection

I successfully collected the data in the simulated environment.

```
user@Debian-KevinPan > /media/user/ext4Storage/癢又勿/Syncing/大學/大三下/AiCapstone/ai-capstone26/hw2 > | main
ls -l data_collection/first_floor
total 72K
dr-xr-xr-x  5 user user  4.0K Apr  8 23:34 .
drwxrwxr-x 10 user user  4.0K Apr  9 07:16 ..
dr-xr-xr-x  2 user user  12K Apr  8 23:34 depth
-r--r--r--  1 user user  25K Apr  8 23:34 GT_pose.npy
dr-xr-xr-x  2 user user  12K Apr  8 23:34 rgb
dr-xr-xr-x  2 user user  12K Apr  8 23:34 semantic

user@Debian-KevinPan > /media/user/ext4Storage/癢又勿/Syncing/大學/大三下/AiCapstone/ai-capstone26/hw2 > | main
ls -l data_collection/first_floor/depth | wc -l
451

user@Debian-KevinPan > /media/user/ext4Storage/癢又勿/Syncing/大學/大三下/AiCapstone/ai-capstone26/hw2 > | main
ls -l data_collection/second_floor
total 68K
dr-xr-xr-x  5 user user  4.0K Apr  9 02:47 .
drwxrwxr-x 10 user user  4.0K Apr  9 07:16 ..
dr-xr-xr-x  2 user user  12K Apr  9 02:47 depth
-r--r--r--  1 user user  24K Apr  9 02:47 GT_pose.npy
dr-xr-xr-x  2 user user  12K Apr  9 02:47 rgb
dr-xr-xr-x  2 user user  12K Apr  9 02:47 semantic

user@Debian-KevinPan > /media/user/ext4Storage/癢又勿/Syncing/大學/大三下/AiCapstone/ai-capstone26/hw2 > | main
ls -l data_collection/second_floor/depth | wc -l
429
```

There is no TODO inside `load.py`, so I didn't modify it.

Task 2: ICP Alignment & 3D Map Reconstruction

I wrote the pipeline (reconstruct()) overview in Task 3, please check there.

1. Unprojection

I implemented it according to the instructions in the template code. Differently in TODO, I added a minus (-) in the y's equation, since we typically treat +y as the direction up in the 3D world, while we treat +v as the direction down (i.e., which row) for images.

Also, I removed those points with depth = 0, which have no depth value and may affect the reconstruction as noise.

```
# I think it should be renamed as RGBD_to_point_cloud, but I'll just keep it
def depth_image_to_point_cloud(rgb_image: str, depth_image: str):
    """
    TASK 1: Geometric Unprojection [cite: 12, 25-27]
    Convert depth pixels (u, v, d) into 3D world points (x, y, z).
    """
    # 1. Read & convert inputs to numpy arrays
    rgb_image = np.array(o3d.io.read_image(rgb_image))
    depth_image = np.array(o3d.io.read_image(depth_image))

    # 2. Convert depth to meters (Habitat depth is often scaled or normalized)
    depth_image = depth_image / DEPTH_SCALE

    # 3. Create a coordinate grid for (u, v) pixels
    u, v = np.meshgrid(np.arange(IMG_W), np.arange(IMG_H))

    # TODO: Implement unprojection logic here
    # 4. unprojection
    x = (u - CX) * depth_image / FX
    y = -(v - CY) * depth_image / FY # for v, up is [-]; for y, up is [+]
    z = -depth_image # (assuming camera looks towards -Z)

    # 5. before point cloud creation
    valid_mask = depth_image.flatten() > 0
    colors_norm = (rgb_image / 255.0).reshape(-1, 3)[valid_mask]
    points_3d = np.stack([
        x.flatten()[valid_mask],
        y.flatten()[valid_mask],
        z.flatten()[valid_mask]
    ], axis=1)

    # 6. create point cloud
    pcd = o3d.geometry.PointCloud()
    pcd.points = o3d.utility.Vector3dVector(points_3d)
    pcd.colors = o3d.utility.Vector3dVector(colors_norm)
    return pcd
```

2. Preprocessing: Downsampling, Estimate Normals, and Compute FPFH

The downsampling code was already written in the template, but I learnt the reason to use it. Then, to estimate normals, the code will use the 30 nearest neighbours in $2.5 \times \text{voxel_size}$ away.

```
def preprocess_point_cloud(pcd, voxel_size):
    """
    Pre-processing: Voxelization and Normal Estimation [cite: 17, 29]
    """

    # 1. Downsampling to
    # - reduce computational cost
    # - remove noise (especially for depth)
    # - make the point cloud more uniform
    pcd_down = pcd.voxel_down_sample(voxel_size)

    # TODO: Estimate normals for pcd_down (required for Point-to-Plane ICP)

    # 2. Estimate normals
    # - Note: Pt-to-Pl ICP is faster than Pt-to-Pt ICP|
    #   ref: https://learnopencv.com/iterative-closest-point-icp-explained/
    radius_estNormal = voxel_size * 2.5
    pcd_down.estimate_normals(
        search_param=o3d.geometry.KDTreeSearchParamHybrid(
            radius=radius_estNormal,
            max_nn=30 # nearest neighbours
        )
    )
```

Then, I use the code directly from the template to fetch fpfh data:

```
# Compute FPFH features for Global Registration [cite: 30]
radius_feature = voxel_size * 5.0
pcd_fpfh = o3d.pipelines.registration.compute_fpfh_feature(
    pcd_down,
    o3d.geometry.KDTreeSearchParamHybrid(
        radius=radius_feature,
        max_nn=100 # nearest neighbours
    )
)
```

But more than that, I do a hack to inject the colour information into FPFH such that RANSAC can be more confident when it matches things correctly. However, if we need to use the code to reconstruct data collected from the real world, the code should convert the RGB space to HSL/HSV etc.: although the environment light sources should not change a lot and fast, shadows may temporarily reduce the lightness, so hue and saturation are the only two reliable factors.

```
# hack: add color infos in fpfh
fpfh_data = np.asarray(pcd_fpfh.data)

color_weight = 50.0
color_data = np.asarray(pcd_down.colors).T * color_weight

enhanced_fpfh_data = np.vstack((fpfh_data, color_data))

enhanced_fpfh = o3d.pipelines.registration.Feature()
enhanced_fpfh.data = enhanced_fpfh_data

return pcd_down, enhanced_fpfh
```

Global Registration (RANSAC)

I wrote a loop that tries 50 times to do RANSAC and ICP. The core logic of execution and validation are in separate functions, `reconstruct__ransac()` and `reconstruct__icp()`. I tried to use `o3d.utility.random.seed()` to make the result deterministic, but it doesn't work. I guess it runs in parallel in background.

```
# 3. Execute Global Registration (RANSAC)
random.seed(666)
attempt = 0
while final_transformation is None and attempt < 50:
    o3d.utility.random.seed(random.randint(0, 100000))
    attempt += 1
    ransac_transformation = reconstruct__ransac(pcd_down, target_pcd_down,
        pcd_fpfh, target_fpfh, camera_poses, failed_attempt_in_seq)
    if ransac_transformation is None:
        continue
    final_transformation = reconstruct__icp(pcd_down, target_pcd_down,
        ransac_transformation, camera_poses, voxel_size,
        failed_attempt_in_seq, args)
```

In `reconstruct__ransac()`, the core is to execute this function. It tries 100000 times to find a match with `confidence=0.9`.

```
def reconstruct__ransac(pcd_down, target_pcd_down, pcd_fpfh, target_fpfh,
                        camera_poses, failed_attempt_in_seq):
    ransac_result = o3d.pipelines.registration.
    registration_ransac_based_on_feature_matching(
        source=pcd_down, target=target_pcd_down,
        source_feature=pcd_fpfh, target_feature=target_fpfh,
        mutual_filter=False,
        max_correspondence_distance=STEP_MOVEMENT_DISTANCE_BOUND *
        (failed_attempt_in_seq + 2),
        # estimation_method=o3d.pipelines.registration.
        TransformationEstimationPointToPoint(False),
        # ransac_n=3,
        checkers=[o3d.pipelines.registration.CorrespondenceCheckerBasedOnEdgeLength
        (0.95)],
        criteria=o3d.pipelines.registration.RANSACConvergenceCriteria
        (max_iteration=100000, confidence=0.9)
    )
```

After that, the result is checked to ensure it's not a bad start point. (See Q2 "Assumption and Validation" for more detailed explanation.)

```
# x.1. Check if the ransac result is good
ransac_translate_dist = np.linalg.norm(
    ransac_result.transformation[:3, 3]
    - camera_poses[-1][:3, 3]
)

ransac_rotate = rotation_between_two_transformation_mat(ransac_result.
transformation, camera_poses[-1])

# x.2. If ransac is good, use it; otherwise, fallback to last pose
if ransac_result.fitness > 0.8 \
    and ransac_translate_dist < STEP_MOVEMENT_DISTANCE_BOUND *
    (failed_attempt_in_seq + 1) \
    and ransac_rotate < ROTATION_BOUND * (failed_attempt_in_seq + 1):

    # i.e., if confident
    # and the distance from last step is small as expected
    # and the rotation is small as expected

    return ransac_result.transformation
else:
    print("- RANSAC failed")
```

In `reconstruct_icp()`, it follows a similar procedure. First it executes `(my_)local_icp_algorithm()` to run the point-to-plane ICP.

```
def reconstruct_icp(pcd_down, target_pcd_down, init_transformation, camera_poses,
                    voxel_size, failed_attempt_in_seq, args):
    icp_transformation = init_transformation

    icp_algo = my_local_icp_algorithm if args.version == 'my_icp' \
    | else local_icp_algorithm

    # start from at least (1+1+1)**0.5 / 2 * voxel_size ~= 0.9 voxelsize
    # end at most 0.04 since 10m / 255 ~= 0.039
    for threshold in np.arange(
        max(0.9 * voxel_size, 0.15),
        min(voxel_size, 0.08),
        -0.01
    ):
        icp_result = icp_algo(
            pcd_down, target_pcd_down,
            icp_transformation,
            threshold=threshold
        )

        # If fitness is 0, it means ICP found absolutely zero correspondences
        if icp_result.fitness == 0:
            print(f"- ICP failed at threshold {threshold}")
            return None

        icp_transformation = icp_result.transformation
```

Then, it will check the result. (Still, see Q2 "Assumption and Validation".)

```
# x.0. if the transformation matrix is the same, it means ICP failed
if np.array_equal(icp_transformation, init_transformation):
    print(f"- ICP result == init_transformation, failed")
    return None

# x.1. get icp result traits
icp_translate_dist = np.linalg.norm(
    icp_transformation[:3, 3]
    - camera_poses[-1][:3, 3]
)
icp_rotate = rotation_between_two_transformation_mat(icp_transformation, camera_poses
[-1])

# x.2. If the result meets assumptions, use it; otherwise, fallback to last pose
if icp_translate_dist < STEP_MOVEMENT_DISTANCE_BOUND * (failed_attempt_in_seq + 1) \
    and icp_rotate < ROTATION_BOUND * (failed_attempt_in_seq + 1):
    return icp_transformation

else:
    print(f"- ICP result is not good, retrying...")
    print(f"-- icp_translate_dist: {icp_translate_dist}")
    print(f"-- icp_rotate: {icp_rotate}")
```

In my code, `local_icp_algorithm()` and `my_local_icp_algorithm()` shares the same parameter format with different logic. The prior directly calls the `o3d`'s point-to-plane ICP function, with 0.001 relative_fitness, 0.001 relative_rmse and 50 iterations at max as the termination criteria.

```
def local_icp_algorithm(source_down, target_down, trans_init, threshold):
    """
    TASK 2: Open3D ICP Implementation (REQUIRED) [cite: 32]
    """
    # TODO: Use o3d.pipelines.registration.registration_icp
    # Estimation method should be TransformationEstimationPointToPlane()

    # trans_init is the initial guess preventing ICP from getting stuck in local minima
    reg_p2p = o3d.pipelines.registration.registration_icp(
        source_down, target_down, threshold, trans_init,
        estimation_method=o3d.pipelines.registration.TransformationEstimationPointToPlane(),
        criteria=o3d.pipelines.registration.ICPConvergenceCriteria(
            max_iteration=50,
            relative_fitness=0.001, # 1e-6 by default
            relative_rmse=0.001 # 1e-6 by default
        )
    )

    return reg_p2p
```

In `my_local_icp_algorithm()`, my code first create a K-D Tree for the search of the target point cloud later.

```
def my_local_icp_algorithm(source_pcd, target_pcd, initial_transform, threshold):
    """
    TASK 2: Custom ICP Implementation (BONUS 20%)
    Implement your own version of Point-to-Plane ICP.
    """

    # TODO: Implement the ICP loop:

    T_global = initial_transform.copy()

    moving_pcd = deepcopy(source_pcd)
    moving_pcd.transform(T_global)

    # 0. Prepare: Create KD Tree
    # - np.asarray is faster than .array, cuz it works w/o copy
    target_tree = o3d.geometry.KDTreeFlann(target_pcd)
    target_points = np.asarray(target_pcd.points)
    target_normals = np.asarray(target_pcd.normals)
```

Then, for each point in the source point cloud, it find the nearest neighbour in the target point cloud through the tree. Here, in order to boost, I tried to preallocate the spaces with `np.zeros()`, but the contribution to the speed is not significant. The code also runs 50 iterations at most.

```
# 1./2./3. ICP loop
max_iteration = 50 # same as in local_icp_algorithm()
for _ in range(max_iteration):
    moving_points = np.asarray(moving_pcd.points)
    num_points = len(moving_points)

    # Pre-allocation
    # valid_points, valid_correspondances, valid_normals = [], [], []
    P_tmp = np.zeros((num_points, 3), dtype=np.float64)
    Q_tmp = np.zeros((num_points, 3), dtype=np.float64)
    N_tmp = np.zeros((num_points, 3), dtype=np.float64)
    valid_count = 0

    # 1. Find nearest neighbors using target_tree.search_knn_vector_3d
    for point in moving_points:
        # return: count, indices[], squared_distances[]
        cnt, idxs, dist_sqr = target_tree.search_knn_vector_3d(point, 1)
        if cnt == 1 and dist_sqr[0] < threshold**2:

            # valid_points.append(point)
            # valid_correspondances.append(target_points[idxs[0]])
            # valid_normals.append(target_normals[idxs[0]])

            # fill np.ndarray directly
            P_tmp[valid_count] = point
            Q_tmp[valid_count] = target_points[idxs[0]]
            N_tmp[valid_count] = target_normals[idxs[0]]
            valid_count += 1
```

```
# translation [x, y, z] + rotation [roll, pitch, yaw]
# = 6 degrees of freedom
if valid_count < 6:
    break

# Finally, slice the array to get only valid data
# - This is a pointer operation, not a copy and thus is quick
# P = np.array(valid_points)
# Q = np.array(valid_correspondances)
# N = np.array(valid_normals)
P = P_tmp[:valid_count]
Q = Q_tmp[:valid_count]
N = N_tmp[:valid_count]
```

Then, I built and solved the linear system $Ax=b$. I didn't use the $A^T Ax=A^T b$ formula provided by the code template. Instead, I simply use `np.linalg.lstsq()`, because it use other methods that don't directly get the inverse of $A^T A$, which is more stable for ill-conditioned matrices. The formulae derivation is collapsed in the code, encapsulated with an empty `{}`.

```
# 2. Build the linear system **Ax = b**

{ ...

# - Create Jacobian matrix (mat_A) using vectorized cross product
A = np.hstack((np.cross(P, N), N))
b = np.sum((Q - P) * N, axis=1)

# 3. Solve for pose update and update T_global
# - Use NumPy's least square method to solve for pose update
# (alternative to (A^T @ A) x = A^T b)
x, _, _, _ = np.linalg.lstsq(A, b, rcond=None)

# Convert x back to dT
dT = np.eye(4)
dT[:3, :3] = R.from_rotvec(x[:3]).as_matrix()
dT[:3, 3] = x[3:]

# Accumulate and apply dT
T_global = dT @ T_global
moving_pcd.transform(dT)

if np.linalg.norm(x) < 1e-3:
    break
```

Then, a result mimicking open3d's is generated and returned.

```
# result.fitness = len(valid_points) / len(source_pcd.points) if len
(source_pcd.points) > 0 else 0.0
result.fitness = valid_count / len(source_pcd.points) if len
(source_pcd.points) > 0 else 0.0
return result
```

Task 3: Camera Pose Estimation & Visualization

Scene: reconstruct() Overview & Cloud Accumulation & Ceiling Cut

In my code, I have two voxel_sizes: one is for the registration process, and the other is for displaying. First I load the files and sort them with the number (note: the sorting key function is added, different from the template.)

```
def reconstruct(args):
    voxel_size = 0.15
    voxel_size_for_display = 0.02
    rgb_dir = os.path.join(args.data_root, "rgb")
    depth_dir = os.path.join(args.data_root, "depth")

    rgb_files = sorted(
        glob.glob(os.path.join(rgb_dir, "*.png")),
        key=lambda x: int(os.path.splitext(os.path.basename(x))[0])
    )
    depth_files = sorted(
        glob.glob(os.path.join(depth_dir, "*.png")),
        key=lambda x: int(os.path.splitext(os.path.basename(x))[0])
    )
```

The Load Ground Truth Poses code is omitted since it's the same as the template. Then, I maintain two point clouds: one is for registration, one is for display.

```
camera_poses = [np.eye(4)]
accumulated_pcd = o3d.geometry.PointCloud()
accumulated_for_display_pcd = o3d.geometry.PointCloud()

# Reconstruction Loop [cite: 29-30]
target_pcd = depth_image_to_point_cloud(rgb_files[0], depth_files[0])
accumulated_for_display_pcd += target_pcd
target_pcd_down, target_fpfh = preprocess_point_cloud(target_pcd, voxel_size)
accumulated_pcd += target_pcd_down
```

Then, the reconstruct loop calls each function mentioned in Task 2 for each frame.

```
failed_attempt_in_seq = 0
for i in range(1, len(rgb_files)):
    print(f"Processing Frame {i}...")
    # TODO: Implement the full pipeline:
    # 1. Convert RGB-D to PointCloud (Task 1)
    pcd = depth_image_to_point_cloud(rgb_files[i], depth_files[i])

    # 2. Preprocess (Voxel/FPFH/Normals)
    pcd_down, pcd_fpfh = preprocess_point_cloud(pcd, voxel_size)

    # 3/4. Registration
    final_transformation = None

    # 3. Execute Global Registration (RANSAC)
    random.seed(666)
    attempt = 0
    while final_transformation is None and attempt < 50: ...

    # 4. Execute Local Registration (ICP - Task 2)
    if final_transformation is None: ...

    # in case the retrying is failed
    if final_transformation is None:
        failed_attempt_in_seq += 1
        print(f"=== There are {failed_attempt_in_seq} shots ignored in a row ===")
    else:
        failed_attempt_in_seq = 0
```

(5. Update camera_poses is described in next section) After each iteration, it clip a box of point cloud centred at the current position from accumulated_pcd (the current frame is not yet stored inside), merge it with the current frame, and generate target_pcd form next iteration.

```
# 6. prepare for next iteration
if final_transformation is not None:
    pcd_down_in_global = deepcopy(pcd_down)
    pcd_down_in_global.transform(final_transformation)

target_pcd = cut_pcd_by_box(
    accumulated_pcd,
    camera_pose[:3, 3],
    radius=1 * (failed_attempt_in_seq + 1)
) + pcd_down_in_global

target_pcd_down, target_fpfh = preprocess_point_cloud(target_pcd, voxel_size)
```

After that, it add the current point cloud into the two accumulated ones. accumulated_pcd use the down-sampled one, while accumulated_for_display_pcd simply use the original one for now.

In the end of each reconstruction iteration, it optionally downsamples the two clouds to prevent out of memory. I think that's because I takes 400~600 shots when collecting data, which is more than 152, 129 or 213 frames mentioned in homework spec.

```
# 7. accum pcd
if final_transformation is not None:
    accumulated_pcd += pcd_down_in_global
    pcd_in_global = deepcopy(pcd)
    pcd_in_global.transform(final_transformation)
    accumulated_for_display_pcd += pcd_in_global

# 8. optionally downsample accum to prevent OOM
if i % 50 == 0 or i == len(rgb_files) - 1:
    accumulated_pcd = accumulated_pcd.voxel_down_sample(voxel_size)
    accumulated_for_display_pcd = accumulated_for_display_pcd.voxel_down_sample(
        voxel_size_for_display)
```

After the reconstruction, the point cloud will be cut to remove the ceiling. (I also returned the uncut one for easily debugging.)

```
camera_poses = np.stack(camera_poses)
# accumulated_pcd = accumulated_pcd.voxel_down_sample(voxel_size_thres)

# TODO: Post-processing: remove the ceiling [cite: 37]
# remove the ceiling by cutting the y axis for y > 0.5
accumulated_pcd_no_ceiling = cut_pcd_ceiling(accumulated_for_display_pcd, height=0.5)

return accumulated_pcd_no_ceiling, camera_poses, gt_poses, accumulated_for_display_pcd
```

Here are the functions to cut. cut_pcd_by_box() is to get a cube of points from the accumulated_pcd for the next reconstruction iteration, while cut_pcd_ceiling() removes the ceiling. (Height is 0.5 in the end of reconstruct().)

```
def cut_pcd_by_box(pcd, center, radius=2):
    min_bound = center - np.array([radius] * 3)
    max_bound = center + np.array([radius] * 3)
    bbox = o3d.geometry.AxisAlignedBoundingBox(min_bound, max_bound)
    return pcd.crop(bbox)

def cut_pcd_ceiling(pcd, height=0.8):
    bbox = pcd.get_axis_aligned_bounding_box()
    min_bound = bbox.get_min_bound()
    max_bound = bbox.get_max_bound()
    max_bound[1] = height # cut Y (height)
    new_bbox = o3d.geometry.AxisAlignedBoundingBox(min_bound, max_bound)
    return pcd.crop(new_bbox)
```

Trajectory & Camera Pose Estimation

For each shot, if ICP failed, the camera pose estimation directly uses the last pose. Otherwise, it uses the transformation calculated by ICP.

```
# 5. Update camera_poses
# camera_pose = camera_poses[-1] @ local_icp_result.transformation
camera_pose = final_transformation if final_transformation is not None \
    else camera_poses[-1] # fallback to last pose est
camera_poses.append(camera_pose)
```

Later in `visualize_and_evaluate()`, the translation is extracted from the local-to-global transformation matrix. (Ground truth follows the same procedure; omitted in this report.)

```
def visualize_and_evaluate(reconstructed_pcd, predicted_cam_poses, gt_poses, args):
    """
    TASK 3: Evaluation & Visualization [cite: 19, 35-38]
    """
    # 1. Create LineSet for estimated trajectory (Red)
    pred_lines = o3d.geometry.LineSet()

    # Note: We'll use Vec3dVec (3 := the vec is (n, 3). d := the vec is float64)
    #       and Vec2iVec (2 := (n, 2). i := int32) to wrap the data

    # 1.1. Extract the points from predicted_cam_poses
    pred_lines.points = o3d.utility.Vector3dVector(
        predicted_cam_poses[:, :3, 3]
        # each in predicted_cam_poses is a 4x4 homogeneous transformation matrix
        #  ${}^{\text{world}}T_{\text{cam's local coord}}$ 
        # here,
        #   for each cam pose([:]),
        #   extract [x, y, z] axis' ([row=0:3])
        #   translation part ([col=3])
    )

    # 1.2. Connect each pair of points with their indeces
    pred_lines.lines = o3d.utility.Vector2iVector(np.array(
        [ [i, i+1] for i in range(len(predicted_cam_poses)-1) ]
    ))

    # 1.3. Set the color of each line
    pred_lines.colors = o3d.utility.Vector3dVector(
        [ [1, 0, 0] for _ in range(len(predicted_cam_poses)-1) ]
    )
```

Evaluation & Visualization

The Mean L2 distance is calculated as $1/n \sum_{i=1}^n \sqrt{(x_{pred_i} - x_{truth_i})^2}$.

```
# TODO: Calculate Mean L2 Distance between predicted_cam_poses and gt_poses [cite: 38]
# L2 = sqrt(dx^2 + dy^2 + dz^2)
mean_l2_error = np.mean(
    np.sqrt(
        np.sum(
            (predicted_cam_poses[:, 0:3, 3] - gt_poses[:, 0:3, 3]) ** 2, axis=1
        )
    )
)

print(f"Mean L2 distance: {mean_l2_error:.6f} meters")

# 3. Visualization
o3d.visualization.draw_geometries([reconstructed_pcd, pred_lines, gt_lines],
                                   window_name=f"Floor {args.floor} Reconstruction")
return mean_l2_error
```

In the main block, the execution time is displayed (this is the code from template). By the way, I added argument checks to warn the user if there's any typo.

```
if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('-f', '--floor', type=int, default=1)
    parser.add_argument('-v', '--version', type=str, default='open3d', help='open3d or my_icp')
    args = parser.parse_args()

    # Argument checks
    if args.version not in ['open3d', 'my_icp']:
        raise ValueError("Invalid version. Use 'open3d' or 'my_icp'.")
    if args.floor not in [1, 2]:
        raise ValueError("Invalid floor. Use 1 or 2.")

    # Set data root based on floor
    args.data_root = f"data_collection/first_floor/" if args.floor == 1 else f"data_collection/second_floor/"

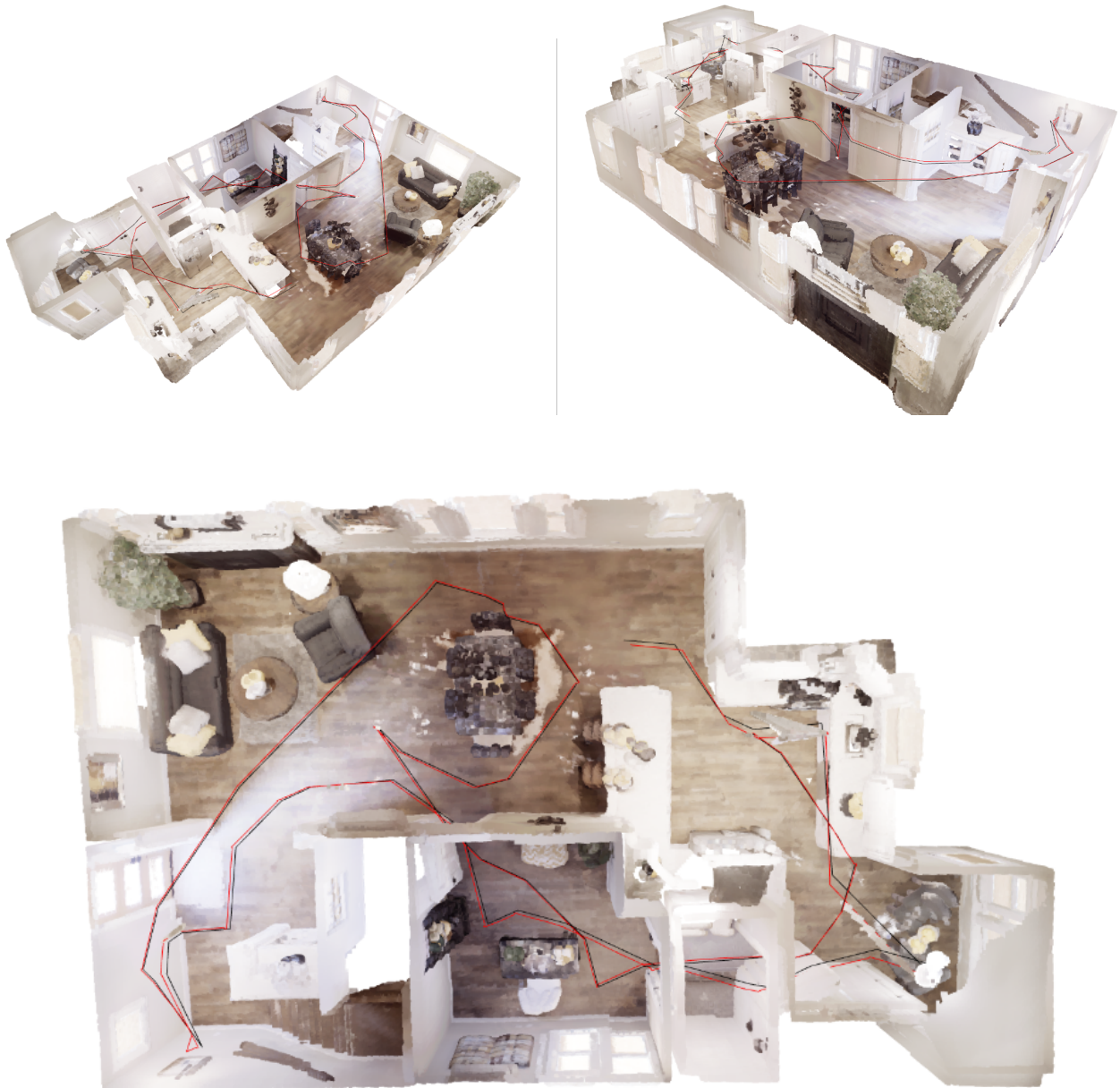
    start_time = time.time()
    result_pcd, pred_poses, gt_poses, result_pcd_w_ceiling = reconstruct(args)

    print(f"Total execution time: {time.time() - start_time:.2f}s") #
    visualize_and_evaluate(result_pcd, pred_poses, gt_poses, args)
    # visualize_and_evaluate(result_pcd_w_ceiling, pred_poses, gt_poses, args)
```

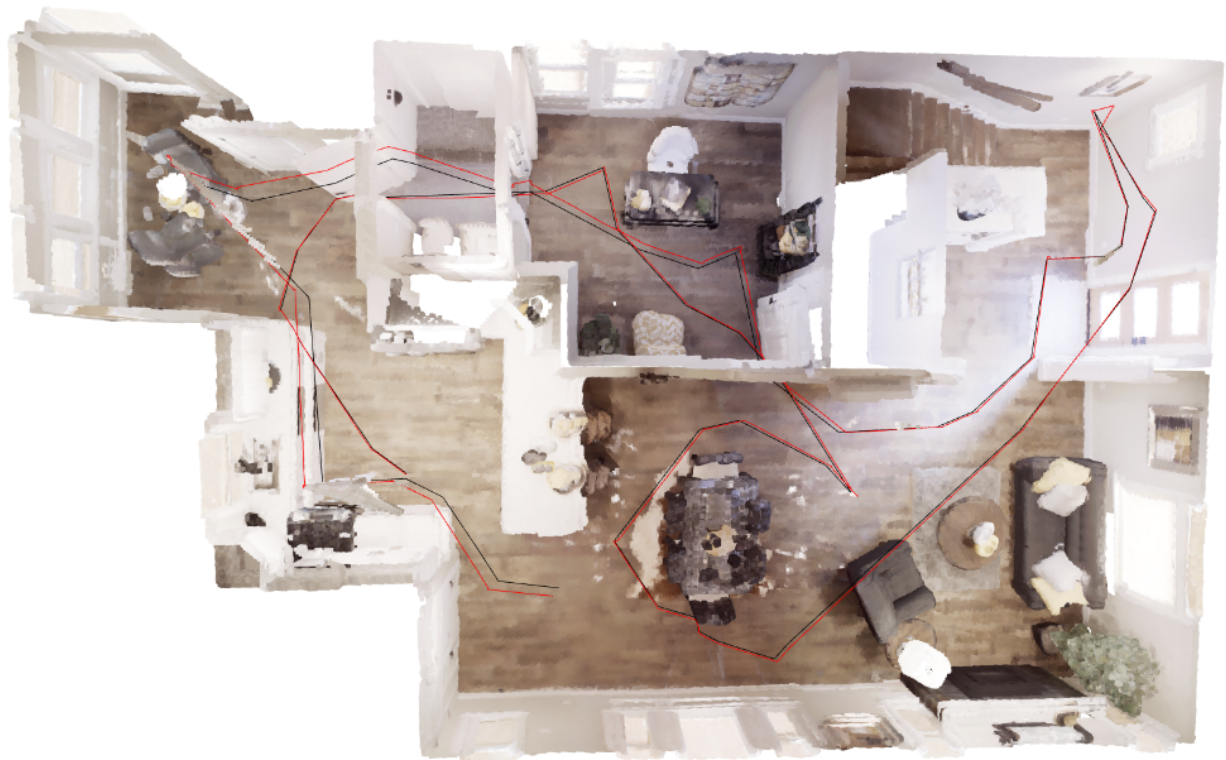
Result & Discussion

Screenshots of apartment_0 Floor 1 reconstruction

Here are the screenshots of my reconstruction. These use the open3d version (Test 1 in the table below):



While these use the my_icp (Test 2 in the table below):



They are reconstructed from 449 different camera shots.

A comparison table (of Hyperparameters and outcomes)

I've tested so many times, but here lists four of them with the same 449-shot collected dataset:

	Test 1	Test 2	Test 3	Test 4
Voxel Size	0.15	0.15	0.15	0.25
Voxel Size for Display	0.02	0.02	0.02	0.02
Normal Estimation	radius=2.5 * voxel_size, max_nn=30			
FPFH Extraction	radius=radius_feature, max_nn=100			
FPFH Color Weight	50			
Tolerance ratio of assumptions	1.5			
Assume Max Movement	0.25m			
Assume Max Rotation	10°			
ICP Version	open3d	my_icp	my_icp	open3d
ICP Convergence Criteria (open3d): [fitness, rmse]	max_iteration=50.			
(my_icp): step size L2 norm	[0.001]*2	0.0001	0.001	[0.001]*2
Multi-step ICP with threshold [start, end, gap]	[	max(0.9 * voxel_size, 0.15), min(voxel_size, 0.08), -0.01		]
RANSAC Random Seed	666			
RANSAC Convergence Criteria	max_iteration=100000, confidence=0.9			
RANSAC Max Retry	50			
Downsample Accum. per {} Iterations	50			
Cut Ceiling Height	0.5			
Execution Time	83.20s	139.96s	200.09s	74.87s
Mean L2 Distance	0.302346m	0.355584m	0.315211m	0.503479
Is visually good?	Yes	Yes	Yes	No (See below)

Discuss the result

Voxel Size and visual result

It took me a long time to achieve a visually good result. When there are more shots, the chance to have a broken reconstruction increase. For example, this is a screenshot from Test 4 in the table above:



As you can see, with a higher voxel_size during registration, the shots fail to align with each other. I didn't find an efficient way to fix it under any voxel size, but I found that voxel_size = 0.15 is a great balance between speed and performance.

my_icp and open3d

To reduce the execution time, I learnt to use `np.asarray()` over `np.array()` or `np.stack()` for `pcd.points`, for it can directly use the data without copy, which obviously boost the progress. Also, I used space pre-allocation with `np.zeros()` to prevent list `.append()` and `np.array()` conversion, but this didn't boost that much as I observe.

So, the final result is,

- To achieve a similar precision as open3d's Point-to-Plane ICP Algorithm, it takes approximately 2.5x the time.
- To achieve a similar speed as open3d's algorithm, mine has a higher (but still tolerable) Mean L2 distance between the trajectory prediction and ground truth.

I think the main reason for this is that, Python itself is a script language that interpretation and type checking cause overheads, which my ICP algorithm implemented upon it also inevitably needs to bear with. In contrast, open3d's algorithm is implemented in C/C++ and compiled beforehand, and thus it's pretty fast.

2. Questions

Q1: What happens if you perform ICP without Global Registration (RANSAC)? Why?

In my experience, it fails when I directly use the last camera pose estimation (transformation) as the initial transformation, which results in a much worse visual output and meter-level Mean L2 distance between the predicted and ground-truth trajectories. I think there mainly are two reasons:

- ICP falls into local minima. An initial value that's not good enough cause ICP trying to align objects with a wrong one.
- Registration without the global information (which is cut to a 1m^3 size from the accumulation point cloud in my code) accumulates the error.

Q2: Describe any tricks used to improve your ICP stability (e.g., Voxel size, Huber loss, etc.).

To increase the stability, I used these tricks:

- Multi-iteration of ICP with Progressive Thresholds:
 - ICP only pair points between which the distances are within a threshold. In my implementation, the threshold shrinks from $\max(0.9 * \text{voxel_size}, 0.15)$ to $\min(\text{voxel_size}, 0.08)$. The result is a much finely aligned world, which is also more stable.
- Aid with global data:
 - If we RANSAC and ICP with only the reconstructed point cloud of last frame, the error accumulates.
 - If we RANSAC and ICP with the whole accumulated point cloud (the world), it will be so slow for sure, and RANSAC will take a long time trying to match points far away from each other if it's not set up correctly.
 - Instead of simply using either of thme, I merge it with a 1m^3 cube of point cloud cut from the accumulated point cloud. This gives RANSAC and ICP more information and increases the probability of successful registration, while controls the execution time in an acceptable range.
- Assumption and Validation: (about translation and rotation)
 - Investigating into load.py, I know that the default step size for "move_forward" is 0.25m , while the default rotation for "turn_left" and "turn_right" is 10° .

- Therefore, I assume the result rotation and translation should be within 1.5x the values, where I call the 1.5x as the tolerance.
- If the RANSAC or ICP result is not within the assumption, my code recalls RANSAC for up to 50 times to get another initial guess and falls back to the last camera_pose estimation (transformation matrix). If both of these don't work, it skips this shot and temporarily increases failed_attempt_in_seq to loosen the tolerance and use more global data.

I believe this is a reasonable operation - in the real world, inertia measurement units (IMU) may be used to estimate the movement and rotation, while technologies like ultra wide band (UWB) can be use to estimate the indoor position with a precision fine to 10cm level.